

Built-in and instance methods

CSC103 Programming Fundamentals / Lecture 14

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Given $x=-8$ and $y=5$, print the larger absolute value using Math methods. Test whether character 7 is a digit.

Calculate the absolute value of each input.

Learning objectives

- Distinguish static and instance method calls
- Explain the conventional main method
- Use Math and Character APIs

CLO-2

Methods as named operations

Static and instance calls

The main entry point

Math and Character

Methods as named operations

- A method has a name, parameters and a result type or void.
- Arguments supply actual input values at a call.
- An API describes what callers may rely on.

Example

```
Math.max(12, 9)  
// name: max  
// arguments: 12 and 9  
// result: 12
```

Static and instance calls

- A static method belongs to a class.
- An instance method operates through an object reference.
- Use the form that the API declares.

Example

```
Math.sqrt(81.0)    // static  
"Java".length()   // instance
```

The main entry point

- The conventional main method is public and static, and returns void.
- `String[] args` receives command-line arguments.
- The JVM can call main without creating a class instance.

Example

```
public static void main(String[] args) {  
    System.out.println("Start");  
}
```

Math and Character

- Math provides operations such as abs, min, max, pow and sqrt.
- Character offers tests such as isDigit and isLetter.
- Check parameter types and exceptional cases in documentation.

Example

```
Math.pow(2, 3)           // 8.0  
Character.isDigit('7')   // true  
Character.toUpperCase('a') // A
```

Reading a method signature

- The parameter list describes the kinds of inputs the method accepts.
- The result type describes the value that a caller receives.
- A static declaration permits a call through the class name.

Example

`Math.sqrt(double a)` returns `double`.
An `int` argument can widen to `double`.
`Math.sqrt(25)` therefore gives `5.0`.

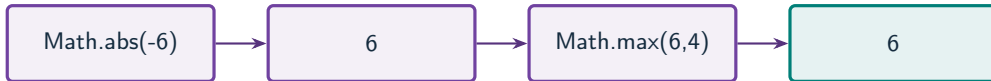
Composing library calls

- The result of one method call can supply an argument to another.
- Java evaluates the inner calls before the outer call receives their values.
- Intermediate variables can make a longer calculation easier to explain.

Example

```
Math.max(Math.abs(-8), Math.abs(5))  
becomes Math.max(8,5), which returns 8.
```

Nested method calls pass results outward



What to notice

An inner call finishes before its returned value is used by the outer call.

Key distinctions

Call	Kind	Input	Result
Math.abs(-8)	Static	int -8	int 8
Math.sqrt(25)	Static	number 25	double 5.0
"Java".length()	Instance	String receiver	int 4
Character.isDigit('7')	Static	char 7	boolean true

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
double distance = Math.sqrt(3 * 3 + 4 * 4);  
char initial = Character.toUpperCase('f');  
System.out.println(distance);  
System.out.println(initial);
```

First example: result

5.0	The main entry point
F	Math and Character
Math.sqrt returns a double. Character.toUpperCase	
returns a char here.	

Guided practice

Given $x=-8$ and $y=5$, print the larger absolute value using Math methods. Test whether character 7 is a digit.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Calculate the absolute value of each input.
- Pass those results to Math.max.
- Use Character.isDigit to classify the supplied character.

The next program uses fixed inputs so its result can be reproduced.

Complete solution

```
public class Solution14 {  
    public static void main(String[] args) throws Exception {  
        int x = -8;  
        int y = 5;  
        int absoluteX = Math.abs(x);  
        int absoluteY = Math.abs(y);  
        int larger = Math.max(absoluteX, absoluteY);  
        System.out.println(larger);  
        System.out.println(Character.isDigit('7'));  
    }  
}
```


Execution trace

Call	Arguments supplied	Returned value
abs	-8	8
abs	5	5
max	8 and 5	8
isDigit	'7'	true

Follow the changing state in execution order.

Solution result

8
true

Why this result follows
Use Character.isDigit to classify the
supplied character.

A common incorrect approach

```
int length = String.length();
```

Example

`length` is an instance method. Use a `String` object such as `"Java".length()`.

Boundary and failure checks

Check the stated input assumptions.

Use Math methods to calculate the hypotenuse for 6 and

8. Read the signatures of every method used.

`Math.max(Math.abs(x),`

`Math.abs(y))` gives 8.

`Character.isDigit('7')` gives true.

Check your understanding

What object is needed for `Math.max`? What does `void` mean?

Write a prediction and explain the rule that supports it.

Answer and explanation

No object is required for this static call. void means the method returns no value.

length is an instance method. Use a String object such as "Java".length().

Compare cases and predict outcomes

Call	Argument meaning	Result
Math.abs(-6)	Magnitude of -6	6
Math.max(6,4)	Larger operand	6
Math.sqrt(36)	Nonnegative square root	6.0

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Compute the distance from (0,0) to (6,8) using `Math.sqrt`. Identify the return type and the arithmetic supplied to it.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
int x = 6;  
int y = 8;  
double distance = Math.sqrt(x * x + y * y);  
System.out.println(distance);
```

- The expression supplies $36 + 64 = 100$.
- `Math.sqrt` returns double, giving 10.0.
- These small inputs fit `int`; large coordinates need an overflow-aware calculation.

Rounding methods are different contracts

- `ceil` moves toward positive infinity; `floor` moves toward negative infinity.
- `rint` chooses the nearest integer with ties to even and returns `double`.
- `round(double)` returns `long`; for halfway cases it rounds toward positive infinity.

Source: supplied ISB campus slides. See speaker notes and [ISB_Source_Map.md](#).

Worked problem: Rounding methods are different contracts

Task

Evaluate rounding at -2.3, 2.5 and 3.5 to expose differences hidden by positive non-tie examples.

Input: Values are fixed in the program for a reproducible demonstration.

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/1)

```
public class IsbLecture14 {  
    public static void main(String[] args) throws Exception {  
        System.out.println(Math.ceil(-2.3));  
        System.out.println(Math.floor(-2.3));  
        System.out.println(Math rint(2.5));  
        System.out.println(Math rint(3.5));  
        System.out.println(Math.round(2.5));  
    }  
}
```

Worked trace and expected result

Call	Result	Interpretation
<code>ceil(-2.3)</code>	-2.0	Toward positive infinity
<code>floor(-2.3)</code>	-3.0	Toward negative infinity
<code>rint(2.5) / round(2.5)</code>	2.0 / 3	Different tie rules

Expected console output

```
-2.0  
-3.0  
2.0  
4.0  
3
```

Extend the ISB example

Practice

Predict `rint(-2.5)` and `round(-2.5)`.

Explain your reasoning

`rint` returns -2.0 (even tie); `round` returns -2 (toward positive infinity at the halfway point).

Lecture summary

- static and instance method calls
- the conventional main method
- Math and Character APIs
- Calculate the absolute value of each input.
- Pass those results to Math.max.
- Use Character.isDigit to classify the supplied character.

After-class practice

Use Math methods to calculate the hypotenuse for 6 and 8. Read the signatures of every method used.

Syllabus reading

Deitel Ch 6

Next lecture: User-defined methods